

计算机组成原理实验报告

基本信息

- 实验名称: Lab3-1
- 姓名: 陈一璟
- 学号: 24300120183

一、实验目的

1. 了解RAM/ROM的基本原理, 掌握调用 Xilinx 库 IP (Block Memory Generator) 实例化 ROM/RAM的方法。
2. 掌握单周期 CPU 中各个控制信号的作用及其生成过程。
3. 掌握单周期 CPU 控制器的工作原理与设计方法, 能够根据指令译码产生相应的控制信号。
4. 理解单周期 CPU 执行指令的过程, 掌握取指阶段和译码阶段的数据通路与控制器的执行流程。
5. 学会在 FPGA 开发板上使用七段数码管和 LED 灯观察指令及控制信号, 验证设计的正确性。

二、实验内容

lab 3-1-1: Head First Register

- 使用 Xilinx Block Memory Generator IP 核构造一个只读存储器 (ROM)。
- 设置 ROM 的数据宽度为 32 位, 深度为 256 (地址线 8 根)。
- 提供一个 COE 文件作为 ROM 的初始值 (包含若干 32 位十六进制数据)。
- 将 ROM 实例化到顶层设计中, 通过板上的 8 个开关提供地址, 将对应地址的数据读出并显示在七段数码管上 (一次显示 8 个十六进制数)。
- 掌握 ROM IP 核的调用、实例化以及上板验证方法。

lab 3-1-2: Register and Controller

- 搭建一个单周期 CPU 的取指和译码阶段电路, 包括以下模块:
 1. **时钟分频器**: 将板载 100MHz 时钟分频为 1Hz, 作为系统主时钟。
 2. **PC 模块**: D 触发器结构, 存储当前指令地址, 复位后从 0 开始, 每个时钟周期更新为 $PC + 4$ 。输出 PC 和指令存储器使能信号 `inst_ce`。
 3. **加法器**: 计算 $PC + 4$ 。
 4. **指令存储器**: 使用 Block Memory Generator IP 配置为 ROM, 加载提供的 `mipstest.coe` 文件 (包含 MIPS 指令机器码)。
 5. **控制器** (`controller.v`):
 - `main_decoder`: 根据指令的高 6 位 `op` 生成 `regwrite`, `regdst`, `alusr`, `branch`, `memwrite`, `memtoreg`, `jump`, `aluop` 等控制信号。
 - `alu_decoder`: 根据 `funct` 和 `aluop` 生成 `alucontrol[2:0]` (ALU 运算类型)。
 - 顶层 `controller` 调用两个 decoder, 输出所有控制信号。
- 将控制信号映射到开发板 LED 灯上。
- 将当前指令显示在七段数码管上。
- 观察随着 1Hz 时钟运行, LED 灯状态和数码管显示按指令顺序变化, 验证控制器逻辑的正确性。

三、实验要求

实验 3-1-1 的要求

- 在 Vivado 中正确添加并配置 Block Memory Generator IP，设置 ROM 模式、32 位数据宽度、256 深度，并加载 COE 文件。
- 编写顶层文件，将 ROM 的地址端口连接到 8 个开关，数据输出连接到七段数码管显示模块。
- 综合、实现、生成比特流，下载到 Basys3 开发板。
- 通过开关改变地址，验证数码管显示的数据与 COE 文件中对应地址的数据一致。
- 将显示结果拍照或截图，写入实验报告。

实验 3-1-2 的要求

- 按照实验原理图搭建完整的取指-译码系统，实现以下模块：
 - `clk_div.v`: 100MHz → 1Hz 分频器。
 - `pc.v`: D 触发器结构，带复位，输出 `pc` 和 `inst_ce`。
 - `controller.v`: 包含 `main_decoder` 和 `alu_decoder`，根据指令产生全部控制信号。
- 指令存储器使用 Block Memory Generator IP，配置为 ROM，加载提供的 `mipstest.coe` 文件。
- 将控制器输出的控制信号按下表连接至板上的 LED 灯（共 11 个 LED）：

控制信号	对应 LED
memtoreg	LED[0]
memwrite	LED[1]
pcsrc	LED[2]
alusr	LED[3]
regdst	LED[4]
regwrite	LED[5]
jump	LED[6]
branch	LED[7]
alucontrol	LED[10:8]

- 将当前指令显示在七段数码管上，支持显示完整指令（32 位）。

四、实验结果

Lab3-1-1结果

1. 模块代码

top.v

- 顶层模块，实例化了 ROM IP 核和显示模块，通过拨码开关选择 ROM 地址并将指令显示在数码管上。

```
module top(  
    input wire      clk,
```

```

input wire      rst,
input wire [7:0] sw,      // 8个拨码开关用于ROM寻址
output wire [7:0] ans,    // 数码管位选
output wire [6:0] seg     // 数码管段选
);
wire [31:0] instruction; // ROM输出的32位指令

// IP核实例化
// 选择 always enable, 没有ena端口
Ins_Rom u_rom (
    .clka(clk),           // 时钟输入
    .addra(sw),           // 地址由拨码开关提供
    .douta(instruction)  // 输出32位指令
);

// 显示模块：将32位指令显示在8个七段数码管上
display u_display (
    .clk(clk),
    .reset(rst),
    .s(instruction),
    .ans(ans),
    .seg(seg)
);

endmodule

```

display.v

- 显示模块，将32位指令显示在8个七段数码管上，实现扫描分频显示（复用之前实验的显示模块）。

```

`timescale 1ns / 1ps

module display(
    input wire clk,reset,
    input wire [31:0]s,
    output wire [6:0]seg,
    output reg [7:0]ans
);
    reg [20:0]count; // 用于时钟分频，原始时钟信号有100MHz
    reg [2:0] sel;   // 数码管选择信号
    reg [3:0] digit; // 要显示的4位数据
    reg [31:0] s_reg; // 存储结果的寄存器

    // 时钟分频
    always @(posedge clk or posedge reset) begin
        if(reset) begin
            count <= 0;
            sel <= 0;
            s_reg <= 0;
        end else begin
            count <= count + 1;

```

```

        if(count == 21'd100000) begin // 100MHz / 100000 = 1kHz
            count <= 0;
            sel <= sel + 1;
            if(sel == 3'd7) begin
                sel <= 0;
                s_reg <= s; // 更新显示数据
            end
        end
    end
end

// 数码管选择和数据显示
always @(*) begin
    case(sel)
        3'd0: begin ans = 8'b11111110; digit = s_reg[3:0]; end // 最低位
        3'd1: begin ans = 8'b11111101; digit = s_reg[7:4]; end
        3'd2: begin ans = 8'b11111011; digit = s_reg[11:8]; end
        3'd3: begin ans = 8'b11110111; digit = s_reg[15:12]; end
        3'd4: begin ans = 8'b11101111; digit = s_reg[19:16]; end
        3'd5: begin ans = 8'b11011111; digit = s_reg[23:20]; end
        3'd6: begin ans = 8'b10111111; digit = s_reg[27:24]; end
        3'd7: begin ans = 8'b01111111; digit = s_reg[31:28]; end // 最高位
        default: begin ans = 8'b11111111; digit = 4'h0; end
    endcase
end

seg7 U4(.din(digit),.dout(seg));
endmodule

```

seg7.v

- 七段数码管译码模块，将4位二进制输入转换为七段显示信号（支持0-F）（复用之前实验的七段数码管译码模块）。

```

`timescale 1ns / 1ps

module seg7(
    input wire [3:0]din,
    output reg [6:0]dout
);

always@(*)
case(din)
    4'b0000: dout=7'b1000000; //0
    4'b0001: dout=7'b1111001; //1
    4'b0010: dout=7'b0100100; //2
    4'b0011: dout=7'b0110000; //3
    4'b0100: dout=7'b0011001; //4
    4'b0101: dout=7'b0010010; //5
    4'b0110: dout=7'b0000010; //6
    4'b0111: dout=7'b1111000; //7

```

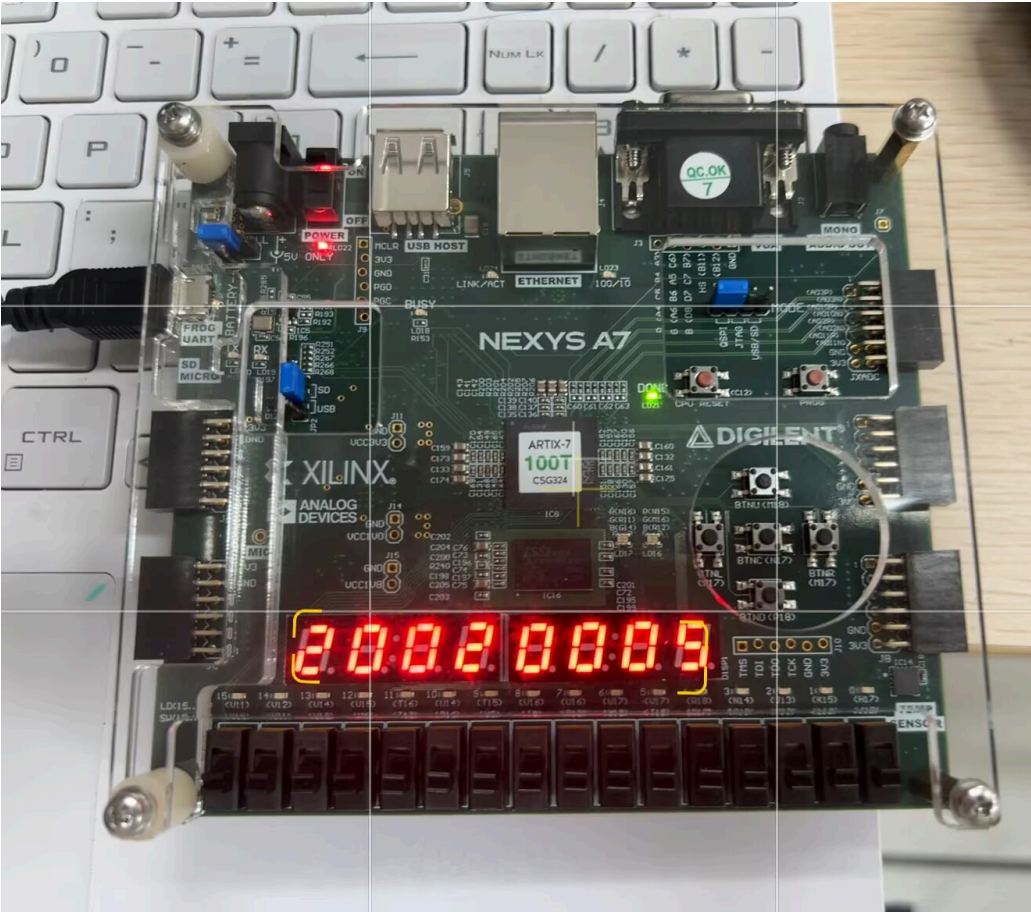
```
4'b1000: dout=7'b0000000; //8
4'b1001: dout=7'b0010000; //9
4'b1010: dout=7'b0001000; //A
4'b1011: dout=7'b0000011; //B
4'b1100: dout=7'b1000110; //C
4'b1101: dout=7'b0100001; //D
4'b1110: dout=7'b0000110; //E
4'b1111: dout=7'b0001110; //F
default: dout=7'b1111111;
endcase

endmodule
```

mipstest.coe (略)

直接使用实验提供的资料文件，共初始化18条指令（0-17）。

2. 实验板结果

拨码开关	指令	实验板显示
00000	20020005	

拨码开关

指令

实验板显示

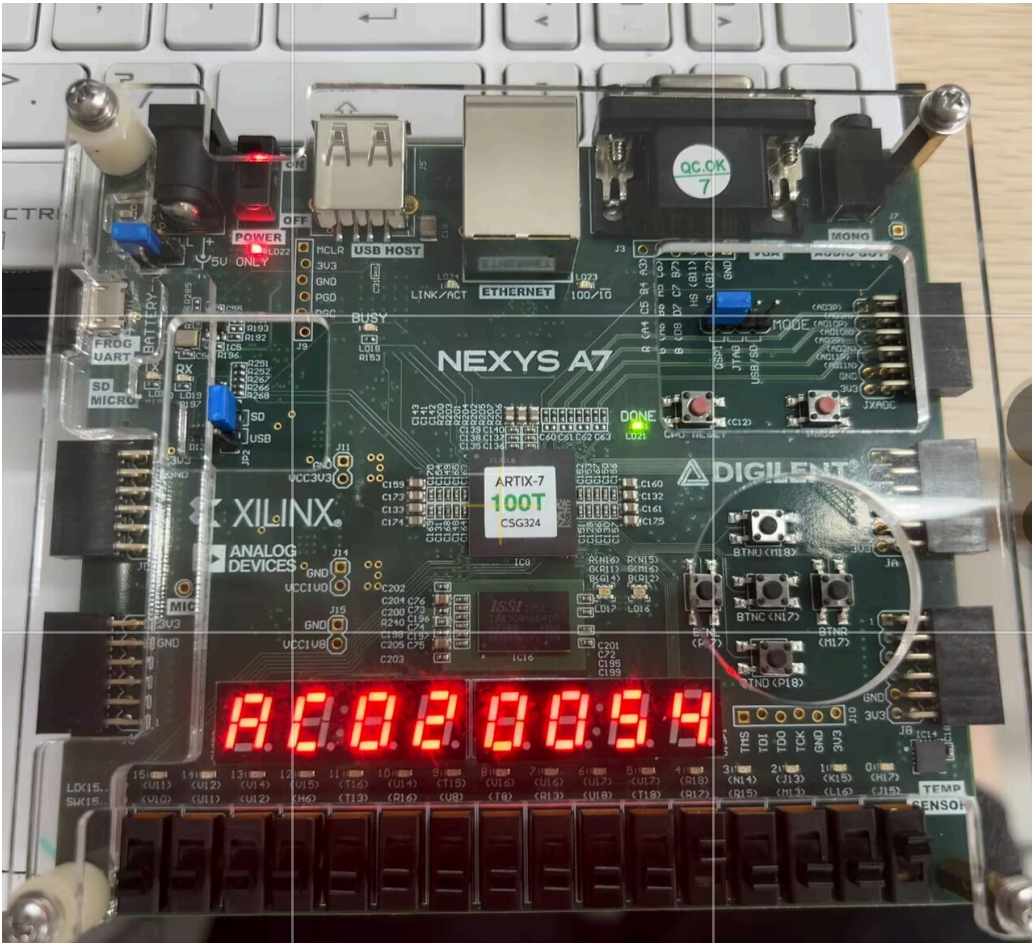
01000

10a7000a



10001

ac020054



实验现象说明：

1. 通过拨码开关选择 ROM 地址，实验板上的 8 个七段数码管会显示对应地址的 32 位 MIPS 指令
2. 动态扫描显示，从左到右依次显示指令的高字节到低字节
3. 图中显示了三个典型地址的指令内容，验证了 ROM 初始化和显示功能的正确性

Lab3-1-2 结果

1. 实验设计

系统包括：时钟分频器、PC 寄存器、加法器、指令存储器、主译码器、ALU 译码器以及七段数码管显示模块。

系统工作流程：

- 100MHz 板载时钟经分频得到 1Hz 时钟。
- PC 在每个时钟上升沿更新为 PC + 4（字节地址）。
- ROM 根据 PC 对应的字地址（PC[9:2]）读出 32 位指令。
- 控制器根据指令的 op[31:26] 和 funct[5:0] 产生所有控制信号。
- 控制信号输出到 LED 灯；指令的低 16 位（或完整 32 位）显示到七段数码管。

特别地，与实验3-1-1不同，在创建IP核时需要选择use ENA pin，以连接PC的inst_ce信号。

2. 实验代码

top.v

- 顶层模块，实例化了时钟分频、PC寄存器、加法器、指令ROM、控制器和显示模块，实现MIPS指令取指和控制信号生成。

```
module top(
    input wire      clk,
    input wire      rst,           // 低电平有效
    output wire [7:0] ans,
    output wire [6:0] seg,
    output wire [10:0] led
);
    wire clk_1hz;
    wire [31:0] pc, pc_next, pc_plus4;
    wire inst_ce;
    wire [31:0] instruction;
    wire [5:0] op, funct;
    wire zero = 1'b0;             // 没有ALU，固定接地
    wire memtoreg, memwrite, pcsrc, alusrc, regdst, regwrite, jump, branch;
    wire [2:0] alucontrol;

    // 将低有效复位转换为高有效复位
    wire rst_sync = ~rst;        // 未按下时 rst_sync=0

    // 时钟分频
    clk_div u_clk_div (
        .clk_100mhz(clk),
```

```
.rst(rst_sync),          // 使用高有效复位
.clk_1hz(clk_1hz)
);

// PC 寄存器
pc u_pc (
    .clk(clk_1hz),
    .rst(rst_sync),      // 使用高有效复位
    .pc_next(pc_next),
    .pc(pc),
    .inst_ce(inst_ce)
);

// 加法器
adder_4 u_adder (
    .pc(pc),
    .pc_plus4(pc_plus4)
);

// PC 下一地址
assign pc_next = pc_plus4;

// Ins_Rom
Ins_Rom u_rom (
    .clka(clk_1hz),
    .ena(inst_ce),      // 供ena pin连接
    .addra(pc[9:2]),    // PC是字节地址，ROM是字地址，需要右移2位
    .douta(instruction)
);

assign op = instruction[31:26];
assign funct = instruction[5:0];

// 控制器
controller u_controller (
    .op(op),
    .funct(funct),
    .zero(zero),
    .memtoreg(memtoreg),
    .memwrite(memwrite),
    .pcsrc(pcsrc),
    .alusrc(alusrc),
    .regdst(regdst),
    .regwrite(regwrite),
    .jump(jump),
    .branch(branch),
    .alucontrol(alucontrol)
);

// 显示指令
display u_display (
    .clk(clk),
    .reset(rst_sync),    // 使用高有效复位
    .s(instruction),
```



```

        .ans(ans),
        .seg(seg)
    );

    assign led[0] = memtoreg;
    assign led[1] = memwrite;
    assign led[2] = pcsrc;
    assign led[3] = alusrc;
    assign led[4] = regdst;
    assign led[5] = regwrite;
    assign led[6] = jump;
    assign led[7] = branch;
    assign led[8] = alucontrol[0];
    assign led[9] = alucontrol[1];
    assign led[10] = alucontrol[2];

endmodule

```

controller.v

- 控制器模块，组合主译码器和ALU译码器，产生所有控制信号。

```

`timescale 1ns / 1ps

module controller(
    input wire[5:0] op, funct,
    input wire zero, // 在top中设置接地
    output wire memtoreg, memwrite,
    output wire pcsrc, alusrc,
    output wire regdst, regwrite,
    output wire jump,
    output wire branch,
    output wire[2:0] alucontrol
);
    wire[1:0] aluop;

    maindec md(op, memtoreg, memwrite, branch, alusrc, regdst, regwrite, jump, aluop);
    aludec ad(funct, aluop, alucontrol);

    assign pcsrc = branch & zero;
endmodule

```

maindec.v

- 主译码器模块，根据指令的op字段生成主要控制信号。

```

`timescale 1ns / 1ps

```

```
module maindec(
    input wire[5:0] op,

    output reg memtoreg,memwrite,
    output reg branch,alusrc,
    output reg regdst,regwrite,
    output reg jump,
    output reg[1:0] aluop
);
always @(*) begin
    memtoreg = 0;
    memwrite = 0;
    branch = 0;
    alusrc = 0;
    regdst = 0;
    regwrite = 0;
    jump = 0;
    aluop = 2'b00;

    case (op)
        6'b000000: begin // R-type
            regwrite = 1; regdst = 1; alusrc = 0;
            branch = 0; memwrite = 0; memtoreg = 0; jump = 0;
            aluop = 2'b10;
        end
        6'b100011: begin // lw
            regwrite = 1; regdst = 0; alusrc = 1;
            branch = 0; memwrite = 0; memtoreg = 1; jump = 0;
            aluop = 2'b00;
        end
        6'b101011: begin // sw
            regwrite = 0; regdst = 0; alusrc = 1;
            branch = 0; memwrite = 1; memtoreg = 0; jump = 0;
            aluop = 2'b00;
        end
        6'b000100: begin // beq
            regwrite = 0; regdst = 0; alusrc = 0;
            branch = 1; memwrite = 0; memtoreg = 0; jump = 0;
            aluop = 2'b01;
        end
        6'b001000: begin // addi
            regwrite = 1; regdst = 0; alusrc = 1;
            branch = 0; memwrite = 0; memtoreg = 0; jump = 0;
            aluop = 2'b00;
        end
        6'b000010: begin // j
            regwrite = 0; regdst = 0; alusrc = 0;
            branch = 0; memwrite = 0; memtoreg = 0; jump = 1;
            aluop = 2'b00; // 无关
        end
        default: ; // 保持默认0
    endcase
end
```

```

    end
endmodule

```

aludec.v

- ALU译码器模块，根据funct字段和aluop信号生成ALU控制信号。

```

`timescale 1ns / 1ps

module aludec(
    input wire[5:0] funct,
    input wire[1:0] aluop,
    output reg[2:0] alucontrol
);
    always @(*) begin
        case (aluop)
            2'b00: alucontrol = 3'b010; // 加法 (lw/sw/addi)
            2'b01: alucontrol = 3'b110; // 减法 (beq)
            2'b10: begin // R-type
                case (funct)
                    6'b100000: alucontrol = 3'b010; // add
                    6'b100010: alucontrol = 3'b110; // sub
                    6'b100100: alucontrol = 3'b000; // and
                    6'b100101: alucontrol = 3'b001; // or
                    6'b101010: alucontrol = 3'b111; // slt
                    default: alucontrol = 3'b010;
                endcase
            end
            default: alucontrol = 3'b010;
        endcase
    end
endmodule

```

pc.v

- 32位程序计数器模块，在每个时钟上升沿更新PC值。

```

`timescale 1ns / 1ps

module pc(
    input wire clk,
    input wire rst,
    input wire [31:0] pc_next, // 下一条指令的地址
    output reg [31:0] pc, // 当前指令地址（输出）
    output wire inst_ce // 指令使能信号，需要连接到ROM的ena pin
);
    assign inst_ce = 1'b1; // 一直使能

```

```
// 在时钟上升沿更新PC值
always @(posedge clk or posedge rst) begin
    if (rst)
        pc <= 32'h0000_0000; // 复位时PC清零，指向程序起始地址
    else
        pc <= pc_next;      // 否则更新为下一条指令地址
    end
endmodule
```

adder_4.v

- PC+4加法器模块，计算下一条顺序指令的地址。

```
`timescale 1ns / 1ps

module adder_4(
    input wire [31:0] pc,          // 当前程序计数器值
    output wire [31:0] pc_plus4   // PC+4（下一条顺序指令地址）
);
    assign pc_plus4 = pc + 32'd4;
endmodule
```

clk_div.v

- 时钟分频模块，将100MHz时钟分频为1Hz，为PC寄存器提供慢速时钟。

```
`timescale 1ns / 1ps

module clk_div(
    input wire clk_100mhz,
    input wire rst,
    output reg clk_1hz
);
    reg [26:0] cnt;
    always @(posedge clk_100mhz or posedge rst) begin
        if (rst) begin
            cnt <= 0;
            clk_1hz <= 0;
        end else if (cnt == 27'd50_000_000 - 1) begin
            cnt <= 0;
            clk_1hz <= ~clk_1hz;
        end else begin
            cnt <= cnt + 1;
        end
    end
endmodule
```

display.v

- 复用自3-1-1实验，代码完全相同。
- 功能：将32位指令显示在8个七段数码管上，实现扫描分频显示。

seg7.v

- 复用自3-1-1实验，代码完全相同。
- 功能：七段数码管译码模块，将4位二进制输入转换为七段显示信号（支持0-F）。

2. 实验板结果

步数	指令类型	亮灯位置（LED编号）
1	addi	9, 5, 3
2	addi	9, 5, 3
3	addi	9, 5, 3
4	or (R)	8, 5, 4
5	and (R)	5, 4
6	add (R)	9, 5, 4
7	beq	10, 9, 7
8	slt (R)	10, 9, 8, 5, 4
9	beq	10, 9, 7
10	addi	9, 5, 3
11	slt (R)	10, 9, 8, 5, 4
12	add (R)	9, 5, 4
13	sub (R)	10, 9, 5, 4
14	sw	9, 3, 1
15	lw	9, 5, 3, 0
16	j	9, 6
17	addi	9, 5, 3
18	sw	9, 3, 1

- LED 编号从 0 到 10，该位为1表示亮灯。
- 通过观察LED灯状态和数码管显示，验证了控制器逻辑的正确性。随着1Hz时钟运行，指令按顺序执行，控制信号和指令显示均符合预期。
- 注：由于 j 指令的 aluop 被设为 00，导致 alucontrol=010，因此 LED9 亮起，不影响功能。
- 视频已经上传到实验报告文件夹下，文件名为 lab3-1-2.mp4。

五、实验思考

1. 遇到的问题及解决方法

1. 问题描述：原本代码的branch信号没有在controller中给输出 解决方法：删除wire branch 并改为 output branch
2. 问题描述：输出端口声明为 wire，在 always @(*) 中赋值生成bitstream时报错 解决方法：将所有输出端口类型从 wire 改为 reg
3. 问题描述：按照lab 3-1-1的步骤，选择的是always enable，因此在lab3-1-2中inst_ce将悬空连接且不驱动任何信号
解决方法：在创建IP核时选择use ENA pin并将ena连接到inst_ce，符合实验要求

2. 实验心得

- 理解了 MIPS 指令取指阶段的完整流程，包括 PC 寄存器更新、指令 ROM 读取、地址对齐（PC[9:2]）等关键步骤。
- 掌握了 MIPS 控制器的设计方法，包括主译码器（根据 op 字段生成控制信号）和 ALU 译码器（根据 funct 字段生成 ALU 控制信号）的分工与协作。
- 学会了时钟高频电路的设计，理解了慢速时钟对于观察指令执行过程的重要性。
- 掌握了 IP 核的配置方法，特别是 ena 引脚的使用场景和连接方式。
- 学会了模块化设计思想，通过分层次、分模块的方式实现复杂系统，提高代码的可读性和可维护性。

六、实验评价

1. 自我评价

将选择的项加粗加斜即可

例如：☐ **优秀** ☐良好 ☐一般 ☐待提高

- 实验完成度：☐ **优秀** ☐良好 ☐一般 ☐待提高
- 掌握程度：☐很好 ☐ **较好** ☐一般 ☐需要加强

2. 实验反馈

1. 实验内容难度：☐偏难 ☐ **适中** ☐偏易
2. 实验时间安排：☐充足 ☐适中 ☐ **紧张**